

NGINX JS DOM — Header and Body Filters

Contents

1. Overview	2
2. Location filter API	2
2.1. <code>location.addHeaderFilter(fn [, opts])</code>	2
2.2. <code>location.addBodyFilter([mode,] fn [, opts])</code>	3
2.3. <code>location.removeHeaderFilter(ref)</code>	4
2.4. <code>location.removeBodyFilter(ref)</code>	4
2.5. <code>location.getHeaderFilter(ref)</code>	4
2.6. <code>location.getBodyFilter(ref)</code>	4
2.7. <code>location.headerFilters</code> (getter)	4
2.8. <code>location.bodyFilters</code> (getter)	5
2.9. Filter options object	5
3. Header filter API	5
4. Body filter API	6
4.1. Whole-body modes (<code>wholeBodySync</code> , <code>wholeBodyAsync</code>)	6
4.2. Streaming modes (<code>streamingSync</code> , <code>streamingAsync</code>)	7
4.2.1. <code>streamingSync</code>	7
4.2.2. <code>streamingAsync</code>	7
4.3. <code>req.sendBuffer(data)</code>	7
4.4. <code>flags</code> object	8
4.5. Mode selection (Mode A vs Mode B)	8
5. Mixed mode (Mode B with streaming filters)	8
6. Execution order and chaining	9
7. Dynamic filter management	9
8. Priority and ordering	10
9. Error handling	10
10. Complete examples	11
10.1. Example 1 — Inject a banner into every HTML response	11
10.2. Example 2 — GZIP-style response size logging	11
10.3. Example 3 — Strip server-sent events containing a keyword	11
10.4. Example 4 — Async translation (<code>wholeBodyAsync</code>)	12
10.5. Example 5 — Async per-chunk watermark injection (<code>streamingAsync</code>)	12
10.6. Example 6 — Chained pipeline: streaming bracket + async uppercase	13
10.7. Example 7 — Priority-based middleware stack	13
10.8. Example 8 — One-shot filter that installs itself then removes itself	14

10.9. Example 9 — Mixed mode: WB normalize then streaming chunk markers	14
10.10. Example 10 — Runtime conditional filter (A/B test)	14
11. Reference summary	15
11.1. Location methods	15
11.2. Body filter modes	16
11.3. <code>req.sendBuffer(data)</code>	16
11.4. Filter options object	16
11.5. <code>flags</code> object (streaming callbacks)	17

1. Overview

NGINX JS DOM lets JavaScript code intercept and transform every HTTP response that passes through a location. The interception happens at two points in nginx's filter chain:

- **Header filters** — called once, just before response headers are sent to the client. The callback receives the live `NginxRequest` object and may mutate response headers in place. The filter is always synchronous.
- **Body filters** — called for each piece of the response body (streaming modes) or once on the complete body (whole-body modes). Five execution modes are available: `wholeBodySync`, `wholeBodyAsync`, `streamingSync`, `streamingAsync`, and `generator` (async generator function).

Filters are registered per-location via methods on the `NginxLocation` object (accessible through `nginx.http.servers[i].locations[j]` or by iterating `nginx.http.servers[i].locations`). Registering a filter from any script that runs at config time — `js_source`, `js_init_http`, `js_preprocess` — is sufficient; the filter then applies to every request that reaches that location.

2. Location filter API

All filter management methods live on the `NginxLocation` object.

2.1. `location.addHeaderFilter(fn [, opts])`

Appends a header-filter callback to the location's header filter list.

```
location.addHeaderFilter(fn)
location.addHeaderFilter(fn, opts)
```

fn

Required. A function with signature `function(req)`. Called once per request just before nginx sends the response headers. The return value is ignored. `req` is a `NginxRequest` — header mutation is described in [Header filter API](#).

opts

Optional object; see [Filter options object](#).

Returns `undefined`. Throws `TypeError` if `fn` is not a function.

2.2. `location.addBodyFilter([mode,] fn [, opts])`

Appends a body-filter callback to the location's body filter list.

```
// Single-argument form – auto-detects async generator functions:
location.addBodyFilter(asyncGenFn)

// Two/three-argument form – explicit mode string:
location.addBodyFilter(mode, fn)
location.addBodyFilter(mode, fn, opts)
```

`mode`

Required string in the two-argument form — one of:

Value	Meaning
<code>'wholeBodySync'</code>	Synchronous whole-body transform. Signature: <code>function(req, body) → string undefined</code> <code>body</code> is the complete accumulated response body as a string. Return a new string to replace it; return <code>undefined</code> (or any non-string) to pass through unchanged.
<code>'wholeBodyAsync'</code>	Asynchronous whole-body transform. Signature: <code>async function(req, body) → Promise<string undefined></code> Same semantics as <code>wholeBodySync</code> but the function may <code>await</code> I/O or timers before returning. If the Promise rejects, the connection is closed with an error.
<code>'streamingSync'</code>	Synchronous per-chunk transform. Signature: <code>function(req, chunk, flags)</code> Called once per nginx body-filter call (one "chunk"). Output is emitted via <code>req.sendBuffer(data)</code> — any number of calls per invocation. If <code>sendBuffer</code> is never called the chunk is dropped (not forwarded).
<code>'streamingAsync'</code>	Asynchronous per-chunk transform. Signature: <code>async function(req, chunk, flags) → Promise<string undefined></code> Called once per nginx body-filter call. The chunk is transformed via the resolved value of the returned Promise: a string replaces the chunk; <code>''</code> drops it; <code>undefined</code> (or any non-string) passes the original chunk through. If the Promise rejects, the connection is closed with an error.

Value	Meaning
'generator'	Async generator transform. Signature: <code>async function*(chunks, req)</code> <code>chunks</code> is a single-element iterable [<code>wholeBodyString</code>]. Iterate with <code>for await (var chunk of chunks)</code> to receive the complete buffered body. <code>yield</code> any number of strings; they are concatenated to form the new body. The generator may <code>await</code> between yields. When omitted, the single-argument form <code>addBodyFilter(asyncGenFn)</code> auto-detects this mode from the function type (<code>AsyncGeneratorFunction</code>).

fn

Required. The filter callback; signature depends on `mode`. In the single-argument form, `fn` must be an async generator function (`async function*`).

opts

Optional object; see [Filter options object](#).

Returns `undefined`. Throws `TypeError` if `mode` is invalid, `fn` is not a function, or the single-argument form is called with a non-async-generator.

2.3. `location.removeHeaderFilter(ref)`

Removes the first header filter whose registered name equals `ref` (string) or whose function reference is `===` to `ref`. No-op if not found.

2.4. `location.removeBodyFilter(ref)`

Removes the first body filter matching `ref` by name or function reference. No-op if not found.

2.5. `location.getHeaderFilter(ref)`

Returns a descriptor object `{ name, priority, fn }` for the first matching header filter, or `null` if not found.

2.6. `location.getBodyFilter(ref)`

Returns a descriptor object `{ name, priority, fn, mode }` for the first matching body filter, or `null` if not found. `mode` is the string passed to `addBodyFilter`.

2.7. `location.headerFilters` (getter)

Returns a snapshot array of `{ name, priority, fn }` objects for all currently registered header filters, in dispatch order.

2.8. `location.bodyFilters` (getter)

Returns a snapshot array of `{ name, priority, fn, mode }` objects for all currently registered body filters, in dispatch order.

2.9. Filter options object

Both `addHeaderFilter` and `addBodyFilter` accept an optional second/third `opts` argument that controls where the new filter is inserted in the list:

```
{
  name:      string,    // symbolic name – used by remove/get
  priority:  number,    // lower = earlier; default 50
  before:    fn | string, // insert immediately before this filter
  after:     fn | string, // insert immediately after this filter
  index:     number     // insert at this absolute position (0-based)
}
```

Precedence (highest first): `index > before / after > priority`.

If `before` or `after` references a filter that does not exist, the new filter is appended.

```
const upper = function(req, body) { return body.toUpperCase(); };

// append at default priority 50
loc.addBodyFilter('wholeBodySync', upper);

// insert with a name so it can be removed later
loc.addBodyFilter('wholeBodySync', upper, { name: 'uppercase' });
loc.removeBodyFilter('uppercase');

// insert before an existing filter
const wrap = function(req, body) { return '[' + body + ']'; };
loc.addBodyFilter('wholeBodySync', wrap, { before: upper });

// insert at the beginning
loc.addBodyFilter('wholeBodySync', wrap, { index: 0 });

// priority-based ordering: lower numbers run first
loc.addBodyFilter('wholeBodySync', upper, { priority: 10 });
loc.addBodyFilter('wholeBodySync', wrap, { priority: 20 });
// → upper runs first, then wrap
```

3. Header filter API

A header filter receives a single `NginxRequest` argument. At that point response headers are still

mutable. Use standard property access on `req`:

```
function(req) {
  // Read a response header
  const type = req.getResponseHeader('content-type');

  // Set/replace a response header
  req.setResponseHeader('x-processed-by', 'nginx-js');

  // Delete a response header
  req.removeResponseHeader('server');

  // Inspect the response status
  const status = req.status; // number, e.g. 200

  // Mutate the status before it is sent
  req.status = 404;
}
```

Header filters are always synchronous. `async function` callbacks are accepted but any awaited operation completes **after** headers are already sent — do not rely on async behaviour inside a header filter.

The return value is ignored.

4. Body filter API

4.1. Whole-body modes (`wholeBodySync`, `wholeBodyAsync`)

The body is accumulated from all nginx chain links until `last_buf` is seen. Then the JS filter chain runs once on the complete body string.

```
// wholeBodySync – transform body synchronously
location.addBodyFilter('wholeBodySync', function(req, body) {
  return body.toUpperCase(); // replace
  // return undefined        → pass through unchanged
});

// wholeBodyAsync – transform body after an async operation
location.addBodyFilter('wholeBodyAsync', async function(req, body) {
  const translated = await callTranslationService(body);
  return translated;      // replace
  // return undefined      → pass through unchanged
  // throw new Error('...') → close the connection
});
```

Chaining: when multiple whole-body filters are registered they run in registration order; each filter

receives the output of the previous one as its **body** argument.

4.2. Streaming modes (**streamingSync**, **streamingAsync**)

Streaming filters run on each chunk individually as nginx delivers it. The filter chain processes chunks in registration order.

4.2.1. **streamingSync**

Output is produced via **req.sendBuffer(data)**. Multiple calls per invocation are allowed; their output is concatenated in call order and forwarded downstream. The next filter in the chain receives the **concatenated output** of the previous filter — not the original chunk.

```
location.addBodyFilter('streamingSync', function(req, chunk, flags) {
    req.sendBuffer(chunk.toUpperCase()); // emit transformed chunk
    // no call to sendBuffer             → chunk is dropped
});
```

flags is an object with one property:

- **flags.last** — **true** if this is the last chunk (corresponds to nginx's **last_buf** flag).

4.2.2. **streamingAsync**

The filter returns a Promise. The resolved value determines the output:

Resolved value	Effect
non-empty string	Replace the chunk with this string
'' (empty string)	Drop the chunk (emit nothing)
undefined / non-string	Pass the chunk through unchanged
Rejection	Close the connection with an error

req.sendBuffer() is **not** the output mechanism for **streamingAsync** — use the return value.

```
location.addBodyFilter('streamingAsync', async function(req, chunk, flags) {
    await nginx.setTimeout(20); // simulate async work
    return chunk.toUpperCase(); // replace
    // return ''                → drop
    // return undefined / no return → pass through
    // throw new Error('...')    → close connection
});
```

4.3. **req.sendBuffer(data)**

Valid only inside a **streamingSync** filter callback. Appends **data** (coerced to string) to the current

chunk's output accumulator. Multiple calls per invocation are allowed.

If called outside a `streamingSync` context (i.e. inside a `wholeBody*` filter or outside any filter) it is a silent no-op and a `WARN` message is written to the error log.

4.4. flags object

Passed as the third argument to both `streamingSync` and `streamingAsync` callbacks:

```
{
  last: boolean    // true when this is the last chunk of the response
}
```

In **mixed mode** (see below) streaming filters receive the whole accumulated body as a single chunk with `flags.last === true`.

4.5. Mode selection (Mode A vs Mode B)

nginx JS DOM uses two internal dispatch strategies based on what filter types are registered for a location:

Mode	When used	Behaviour
Mode A — streaming	Location has no <code>wholeBody*</code> filters	nginx body-filter calls pass directly to the streaming chain; the body is never accumulated. Each call to nginx's body filter = one call to each streaming filter.
Mode B — whole-body	Location has at least one <code>wholeBody*</code> filter	The body is fully accumulated, then ALL filters run in registration order on the complete body. <code>streamingSync/streamingAsync</code> filters in this list receive the whole body as a single last chunk.

The mode is selected automatically based on the filter list at request time. There is no explicit configuration knob.

5. Mixed mode (Mode B with streaming filters)

When both `wholeBody*` and `streaming*` filters share a location, Mode B is used and all filters run in registration order on the accumulated body. Streaming filters behave as follows:

- `streamingSync`: called with `(req, body, {last: true})`. Emit output via `req.sendBuffer()`; no call → body is dropped.
- `streamingAsync`: called with `(req, body, {last: true})`. Resolved string replaces the body; `undefined` passes through; `''` drops it.

```
// Filter 1 (wholeBody): uppercase the body
location.addBodyFilter('wholeBodySync', function(req, body) {
  return body.toUpperCase();
});

// Filter 2 (streaming): wrap in brackets – runs after filter 1
location.addBodyFilter('streamingSync', function(req, chunk, flags) {
  req.sendBuffer('[' + chunk + ']');
});

// Request body 'hello' → 'HELLO' → '[HELLO]'
```

6. Execution order and chaining

Filters execute in the order they appear in the location's filter list. Both header and body filter lists are independent.

```
// Header filter runs first, body filters run in order for each request
location.addHeaderFilter(function(req) {
  req.setResponseHeader('x-stage', 'raw');
});

const step1 = function(req, body) { return body.trim(); };
const step2 = function(req, body) { return body.toUpperCase(); };
const step3 = function(req, body) { return '[' + body + ']'; };

location.addBodyFilter('wholeBodySync', step1);
location.addBodyFilter('wholeBodySync', step2);
location.addBodyFilter('wholeBodySync', step3);
// body passes through step1 → step2 → step3 in that order
```

7. Dynamic filter management

Filters can be added and removed at runtime from within a running filter callback. The implementation uses copy-on-write (COW) for the filter list: mutations during dispatch do not affect the current iteration but take effect for the next request.

```
let patched = false;
const firstRequest = function(req, body) {
  if (!patched) {
    // Add a second filter on the first request; takes effect on second
    location.addBodyFilter('wholeBodySync', function(req, body) {
      return body + ' (patched)';
    });
    patched = true;
  }
}
```

```

    }
    return body;
  };
  location.addBodyFilter('wholeBodySync', firstRequest);

```

```

// Remove a named filter while it is running (safe – COW guarantees
// the current dispatch sees the old list)
const selfRemove = function(req, body) {
  location.removeBodyFilter('once'); // removes itself
  return body + ' [one-shot]';
};
location.addBodyFilter('wholeBodySync', selfRemove, { name: 'once' });

```

8. Priority and ordering

Filters added without an explicit **index**, **before**, or **after** option are inserted based on **priority** (lower = earlier, default **50**):

```

// priority 10 runs before priority 20 runs before priority 50 (default)
location.addBodyFilter('wholeBodySync',
  (req, body) => body.trimStart(), { priority: 10 });

location.addBodyFilter('wholeBodySync',
  (req, body) => body.toUpperCase(), { priority: 20 });

location.addBodyFilter('wholeBodySync',
  (req, body) => '[' + body + ']'); // priority 50 (default)
// Order: trimStart → toUpperCase → bracket

```

9. Error handling

Situation	Behaviour
wholeBodySync filter throws	Exception is logged; body is passed through unchanged; next filter runs.
wholeBodyAsync filter rejects	Error is logged; connection is closed (HTTP 500 or connection reset).
streamingSync filter throws	Exception is logged; chunk is dropped; next filter runs.
streamingAsync filter rejects	Error is logged; connection is closed.
addBodyFilter / addHeaderFilter called with wrong type	TypeError is thrown immediately (synchronous, propagates to the caller).

10. Complete examples

10.1. Example 1 — Inject a banner into every HTML response

```
(function() {
  var loc = nginx.http.servers[0].locations.find(
    function(l) { return l.path === '/'; }
  );

  // Modify the Content-Type header to ensure UTF-8
  loc.addHeaderFilter(function(req) {
    var ct = req.getResponseHeader('content-type') || '';
    if (ct.indexOf('charset') === -1 && ct.indexOf('text/html') !== -1) {
      req.setResponseHeader('content-type', ct + '; charset=utf-8');
    }
  });

  // Inject a banner comment before </body>
  loc.addBodyFilter('wholeBodySync', function(req, body) {
    var banner = '<!-- served by nginx-js -->';
    return body.replace('</body>', banner + '</body>');
  });
})();
```

10.2. Example 2 — GZIP-style response size logging

```
(function() {
  var loc = nginx.http.servers[0].locations[0];
  var totalBytes = 0;

  loc.addBodyFilter('wholeBodySync', function(req, body) {
    totalBytes += body.length;
    nginx.log('info', 'Response size: ' + body.length + ' bytes '
      + '(cumulative: ' + totalBytes + ')');
    return undefined; // pass through unchanged
  });
})();
```

10.3. Example 3 — Strip server-sent events containing a keyword

```
(function() {
```

```

var loc = nginx.http.servers[0].locations.find(
    function(l) { return l.path === '/events/'; }
);

// Per-chunk streaming: drop any SSE event containing 'internal'
loc.addBodyFilter('streamingSync', function(req, chunk, flags) {
    if (chunk.indexOf('internal') !== -1) {
        return; // no sendBuffer → drop this chunk
    }
    req.sendBuffer(chunk);
});
})();

```

10.4. Example 4 — Async translation (wholeBodyAsync)

```

(function() {
    var loc = nginx.http.servers[0].locations.find(
        function(l) { return l.path === '/translate/'; }
    );

    loc.addBodyFilter('wholeBodyAsync', async function(req, body) {
        // nginx.setTimeout returns a Promise that resolves after ms
        await nginx.setTimeout(0); // yield to event loop

        // Simulate async translation (replace with a real upstream fetch)
        var translated = body.split('hello').join('hola');
        return translated;
    });
})();

```

10.5. Example 5 — Async per-chunk watermark injection (streamingAsync)

```

(function() {
    var loc = nginx.http.servers[0].locations.find(
        function(l) { return l.path === '/stream/'; }
    );

    loc.addBodyFilter('streamingAsync', async function(req, chunk, flags) {
        await nginx.setTimeout(0); // yield to event loop

        if (flags.last) {
            // Append a watermark to the last chunk
            return chunk + '\n<!-- end of stream -->';
        }
    });
})();

```

```
        return chunk;    // pass through other chunks unchanged
    });
})();
```

10.6. Example 6 — Chained pipeline: streaming bracket + async uppercase

```
(function() {
    var loc = nginx.http.servers[0].locations[0];

    // Step 1: synchronously wrap each chunk in angle brackets
    loc.addBodyFilter('streamingSync', function(req, chunk, flags) {
        req.sendBuffer('<' + chunk + '>');
    });

    // Step 2: asynchronously uppercase (receives step 1 output as input)
    loc.addBodyFilter('streamingAsync', async function(req, chunk, flags) {
        await nginx.setTimeout(5);
        return chunk.toUpperCase();    // '<hello>' → '<HELLO>'
    });
})();
```

10.7. Example 7 — Priority-based middleware stack

```
(function() {
    var loc = nginx.http.servers[0].locations[0];

    // Security: strip internal headers – run first (priority 10)
    loc.addHeaderFilter(function(req) {
        req.removeResponseHeader('x-internal-token');
        req.removeResponseHeader('x-debug-info');
    }, { priority: 10 });

    // Observability: add timing header – run last (priority 90)
    var started = Date.now();
    loc.addHeaderFilter(function(req) {
        req.setResponseHeader('x-response-ms',
                               String(Date.now() - started));
    }, { priority: 90 });

    // Body: redact PII – normal priority (50)
    loc.addBodyFilter('wholeBodySync', function(req, body) {
        return body.replace(/\b\d{4}-\d{4}-\d{4}-\d{4}\b/g, '****-****-****-****');
    });
})();
```

10.8. Example 8 — One-shot filter that installs itself then removes itself

```
(function() {
    var loc = nginx.http.servers[0].locations[0];

    // Fires only on the first request, then removes itself
    loc.addBodyFilter('wholeBodySync', function firstOnly(req, body) {
        loc.removeBodyFilter(firstOnly);
        return '<!-- first request -->\n' + body;
    }, { name: 'first-only' });
})();
```

10.9. Example 9 — Mixed mode: WB normalize then streaming chunk markers

```
(function() {
    var loc = nginx.http.servers[0].locations.find(
        function(l) { return l.path === '/api/'; }
    );

    // Filter 1 (wholeBodySync): parse JSON and re-serialize normalized
    loc.addBodyFilter('wholeBodySync', function(req, body) {
        try {
            var obj = JSON.parse(body);
            return JSON.stringify(obj); // compact, keys sorted by JS engine
        } catch (e) {
            return undefined; // pass through if not JSON
        }
    });

    // Filter 2 (streamingAsync): add a Content-Length annotation asynchronously
    // Receives the normalized body as a single chunk (flags.last === true)
    loc.addBodyFilter('streamingAsync', async function(req, chunk, flags) {
        await nginx.setTimeout(0);
        return chunk + '\n/* length: ' + chunk.length + ' */';
    });
})();
```

10.10. Example 10 — Runtime conditional filter (A/B test)

```
(function() {
    var loc = nginx.http.servers[0].locations[0];
```

```

var featureEnabled = false;

// Toggle can be flipped externally via a SharedWorker or another
// request handler that has access to the same closure.
function enableFeature() { featureEnabled = true; }
function disableFeature() { featureEnabled = false; }

loc.addBodyFilter('wholeBodySync', function(req, body) {
    if (!featureEnabled) {
        return undefined; // pass through when A/B flag is off
    }
    return '<div class="beta">' + body + '</div>';
});

// Expose control surface via a separate management location
var mgmt = nginx.http.servers[0].locations.find(
    function(l) { return l.path === '/admin/feature/'; }
);
if (mgmt) {
    mgmt.handler = function(req) {
        var enable = req.uri.indexOf('?enable') !== -1;
        if (enable) { enableFeature(); } else { disableFeature(); }
        req.respond(200, {}, 'feature ' + (featureEnabled ? 'ON' : 'OFF'));
    };
}
})();

```

11. Reference summary

11.1. Location methods

Method	Returns	Description
<code>addHeaderFilter(fn [, opts])</code>	<code>undefined</code>	Register a header-filter callback.
<code>addBodyFilter(mode, fn [, opts])</code>	<code>undefined</code>	Register a body-filter callback.
<code>removeHeaderFilter(ref)</code>	<code>undefined</code>	Remove a header filter by name or function reference.
<code>removeBodyFilter(ref)</code>	<code>undefined</code>	Remove a body filter by name or function reference.
<code>getHeaderFilter(ref)</code>	<code>{name, priority, fn}</code> or <code>null</code>	Retrieve header filter descriptor.
<code>getBodyFilter(ref)</code>	<code>{name, priority, fn, mode}</code> or <code>null</code>	Retrieve body filter descriptor.

Method	Returns	Description
<code>headerFilters</code> (<i>getter</i>)	Array of { <code>name</code> , <code>priority</code> , <code>fn</code> }	Snapshot of all header filters in dispatch order.
<code>bodyFilters</code> (<i>getter</i>)	Array of { <code>name</code> , <code>priority</code> , <code>fn</code> , <code>mode</code> }	Snapshot of all body filters in dispatch order.

11.2. Body filter modes

Mode string	Signature	Output mechanism
<code>'wholeBodySync'</code>	<code>fn(req, body) → string undefined</code>	Return value
<code>'wholeBodyAsync'</code>	<code>async fn(req, body) → Promise<string undefined></code>	Resolved value of returned Promise
<code>'streamingSync'</code>	<code>fn(req, chunk, flags)</code>	<code>req.sendBuffer(data)</code> — zero or more calls
<code>'streamingAsync'</code>	<code>async fn(req, chunk, flags) → Promise<string undefined></code>	Resolved value of returned Promise

11.3. `req.sendBuffer(data)`

Parameter	Description
<code>data</code>	String (or coercible to string) to append to the current chunk output.

Valid only inside a `streamingSync` callback. No-op + **WARN** log in all other contexts.

11.4. Filter options object

Property	Type	Description
<code>name</code>	string	Symbolic name for later <code>remove</code> / <code>get</code> calls. Optional.
<code>priority</code>	number	Insertion priority. Lower = earlier. Default <code>50</code> . Used when no positional option (<code>index</code> , <code>before</code> , <code>after</code>) is given.
<code>index</code>	number	Absolute 0-based insertion position. Takes precedence over all other options.
<code>before</code>	function or string	Insert immediately before this filter (matched by function reference or name).
<code>after</code>	function or string	Insert immediately after this filter (matched by function reference or name).

11.5. **flags** object (streaming callbacks)

Property	Type	Description
last	boolean	true if this is the final chunk of the response body.